

Miniproyecto de comunicación entre ESP-32

->RILB ###NOTA, no tengo suficientes componentes entonces algunos códigos parecieran diferentes pero sigue siendo lo mismo para algunos e intentare hacer algunos usos más sencillos.

Usando Displays OLED y botones (para algunos Joysticks) en cada uno de los microcontroladores, cumpliremos uno de los objetivos de este gran proyecto, comunicar varias ESP entre si y que se pueda saber quien mando el mensaje y cual es contenido del mensaje.

Teniendo en cuenta que el protocolo ESP-NOW nos permite comunicar de forma casi inmediata 2 o más ESP, dependiendo de cuantas se necesiten.

Lógica de comunicación.

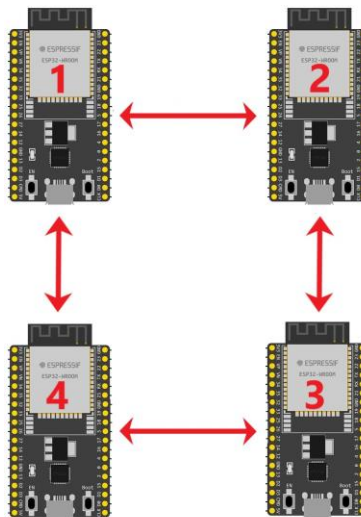
ESP-32 "1" → Envía a **ESP-32 "2"** y a **ESP-32 "4"** → y retransmite de **ESP-32 "2"** y de **ESP-32 "4"**.

ESP-32 "2" → Envía a **ESP-32 "1"** y a **ESP-32 "3"** → y retransmite de **ESP-32 "1"** y de **ESP-32 "3"**.

ESP-32 "3" → Envía a **ESP-32 "2"** y a **ESP-32 "4"** → y retransmite de **ESP-32 "2"** y de **ESP-32 "4"**.

ESP-32 "4" → Envía a **ESP-32 "1"** y a **ESP-32 "3"** → y retransmite de **ESP-32 "1"** y de **ESP-32 "3"**.

Cada ESP recibirá mensajes de todas las demás y los mostrará en su pantalla OLED.



###NOTA: esta lógica de comunicación y el como esta hecho el código no tiene un servidor central, solo está hecho para que todos reciban los mensajes de un solo ESP, el cual podría ser cualquiera que este conectado en su red.

Próximos pasos

Definir el formato de los mensajes:

Como aún no colocamos ningún teclado numérico ni nada, usaremos un Joystick para seleccionar los mensajes que queremos, para este caso cada ESP que reciba el mensaje también mostrará quien lo envió, dando un mensaje como “[ESP x]: Hola”

Agregar lógica para evitar retransmisión infinita

- Para esto usaremos un **ACK (Acknowledgment)**, el cual se usa en redes como una confirmación de recepción. Su propósito es asegurarse de que un mensaje enviado realmente fue recibido por el destinatario.
- ¿Cómo funciona? - Cuando un ESP32 envía un mensaje (por ejemplo, "ESP2:Holi"), lo manda a todos sus vecinos. Cada nodo que recibe ese mensaje responde con un ACK tipo: 'Recibio??:\nESP3'. Este mensaje de confirmación lo envía solo al remitente original, no a todos.
- El nodo que envió el mensaje (por ejemplo, ESP2) escucha si recibió esos ACKs durante un pequeño tiempo (~1 segundo) y guarda si cada vecino respondió o no.
- Finalmente, ESP2 muestra en pantalla algo como:

Recibio??:

ESP1 Si

ESP3 NO

ESP4 Si

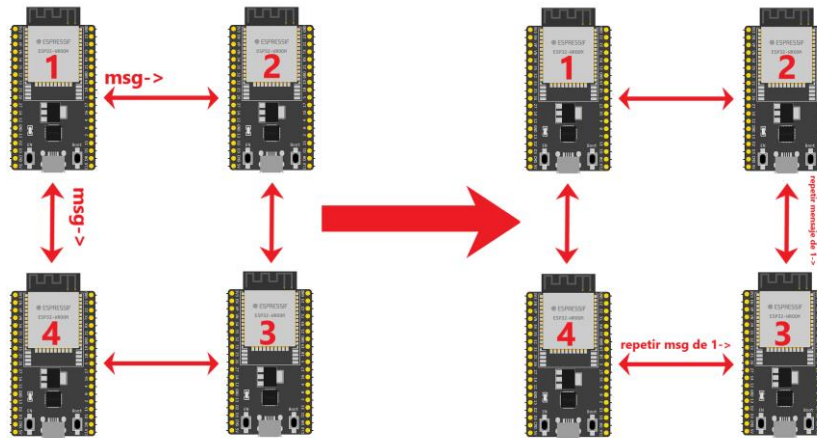
###NOTA: se usa el ACK con el mensaje ‘se recibió??’, además de que el ACK no es reenviado, el código lo retiene con la parte: ‘if not rec_text.startswith("Recibio??:\n"):'

ESP 1 MAC: b'\x9c\x9c\x1f\xc8\x07\xc4'	----la de Joel
ESP 2 MAC: b'\x08\xb6\x1f7\xd6\l'	----de LAB
ESP 3 MAC: b'\xa0\xb7eb\xef\xa4'	---- la de Lucero
ESP4 MAC: b'\x94<\xc62\xca0'	----la RILB

Se busca que en las ESP se integren las siguientes funciones:

1. Repetidor:

- Si ESP2 recibe un mensaje desde ESP1, lo reenvía a ESP3, y viceversa.



2. .1 -Envío con joystick:

- Se usa el eje X de un joystick (conectado a un pin ADC, por ejemplo, GPIO34) para cambiar entre tres mensajes predefinidos.
- El botón del joystick (por ejemplo, conectado a GPIO15) se usa para enviar el mensaje seleccionado.
- *Nota:* La mayoría de los módulos de joystick funcionan alimentados a 3.3 V, pero te recomiendo revisar las especificaciones de tu módulo. En general, si tu módulo es del tipo “analógico” (por ejemplo, KY-023 o similar), puedes alimentarlo con 3.3 V sin problema.

.2 -Envío con botones:

- Un botón (conectado de un GPIO, por ejemplo, GPIO02) para cambiar entre tres mensajes predefinidos.
- Un botón (a otra terminal GPIO, como GPIO04) se usa para enviar el mensaje seleccionado.

3. Display OLED:

- Se utiliza un Display (o OLED) para mostrar el menú actual de mensajes por enviar, sí el mensaje se envió y mensajes recibidos en formato –[Remitente]: Mensaje–.

Código para el protocolo I2C en el Display LCD 128x64 “ssd1306.py”

#Nota, es importante tener este código en la memoria dela ESP, entienda qué es lo que debe mandar al Display, de lo contrario mostrará errores de interpretación o que no se encuentran ciertas funciones.

```
# MicroPython SSD1306 OLED driver, I2C and SPI interfaces
```

```
from micropython import const
import framebuf
```

```

# register definitions
SET_CONTRAST = const(0x81)
SET_ENTIRE_ON = const(0xA4)
SET_NORM_INV = const(0xA6)
SET_DISP = const(0xAE)
SET_MEM_ADDR = const(0x20)
SET_COL_ADDR = const(0x21)
SET_PAGE_ADDR = const(0x22)
SET_DISP_START_LINE = const(0x40)
SET_SEG_REMAP = const(0xA0)
SET_MUX_RATIO = const(0xA8)
SET_IREF_SELECT = const(0xAD)
SET_COM_OUT_DIR = const(0xC0)
SET_DISP_OFFSET = const(0xD3)
SET_COM_PIN_CFG = const(0xDA)
SET_DISP_CLK_DIV = const(0xD5)
SET_PRECHARGE = const(0xD9)
SET_VCOM_DESEL = const(0xDB)
SET_CHARGE_PUMP = const(0x8D)

# Subclassing FrameBuffer provides support for graphics primitives
# http://docs.micropython.org/en/latest/pyboard/library/framebuf.html
class SSD1306(framebuf.FrameBuffer):
    def __init__(self, width, height, external_vcc):
        self.width = width
        self.height = height
        self.external_vcc = external_vcc
        self.pages = self.height // 8
        self.buffer = bytearray(self.pages * self.width)
        super().__init__(self.buffer, self.width, self.height, framebuf.MONO_VLSB)
        self.init_display()

    def init_display(self):
        for cmd in (
            SET_DISP, # display off
            # address setting
            SET_MEM_ADDR,
            0x00, # horizontal
            # resolution and layout
            SET_DISP_START_LINE, # start at line 0
            SET_SEG_REMAP | 0x01, # column addr 127 mapped to SEG0
            SET_MUX_RATIO,
            self.height - 1,
            SET_COM_OUT_DIR | 0x08, # scan from COM[N] to COM0
            SET_DISP_OFFSET,
            0x00,
            SET_COM_PIN_CFG,
            0x02 if self.width > 2 * self.height else 0x12,
            # timing and driving scheme

```

```

    SET_DISP_CLK_DIV,
    0x80,
    SET_PRECHARGE,
    0x22 if self.external_vcc else 0xF1,
    SET_VCOM_DESEL,
    0x30, # 0.83*Vcc
    # display
    SET_CONTRAST,
    0xFF, # maximum
    SET_ENTIRE_ON, # output follows RAM contents
    SET_NORM_INV, # not inverted
    SET_IREF_SELECT,
    0x30, # enable internal IREF during display on
    # charge pump
    SET_CHARGE_PUMP,
    0x10 if self.external_vcc else 0x14,
    SET_DISP | 0x01, # display on
): # on
    self.write_cmd(cmd)
self.fill(0)
self.show()

def poweroff(self):
    self.write_cmd(SET_DISP)

def poweron(self):
    self.write_cmd(SET_DISP | 0x01)

def contrast(self, contrast):
    self.write_cmd(SET_CONTRAST)
    self.write_cmd(contrast)

def invert(self, invert):
    self.write_cmd(SET_NORM_INV | (invert & 1))

def rotate(self, rotate):
    self.write_cmd(SET_COM_OUT_DIR | ((rotate & 1) << 3))
    self.write_cmd(SET_SEG_REMAP | (rotate & 1))

def show(self):
    x0 = 0
    x1 = self.width - 1
    if self.width != 128:
        # narrow displays use centred columns
        col_offset = (128 - self.width) // 2
        x0 += col_offset
        x1 += col_offset
    self.write_cmd(SET_COL_ADDR)
    self.write_cmd(x0)
    self.write_cmd(x1)

```

```
self.write_cmd(SET_PAGE_ADDR)
self.write_cmd(0)
self.write_cmd(self.pages - 1)
self.write_data(self.buffer)
```

```
class SSD1306_I2C(SSD1306):
```

```
    def __init__(self, width, height, i2c, addr=0x3C, external_vcc=False):
        self.i2c = i2c
        self.addr = addr
        self.temp = bytearray(2)
        self.write_list = [b"\x40", None] # Co=0, D/C#=1
        super().__init__(width, height, external_vcc)
```

```
    def write_cmd(self, cmd):
        self.temp[0] = 0x80 # Co=1, D/C#=0
        self.temp[1] = cmd
        self.i2c.writeto(self.addr, self.temp)
```

```
    def write_data(self, buf):
        self.write_list[1] = buf
        self.i2c.writevto(self.addr, self.write_list)
```

```
class SSD1306_SPI(SSD1306):
```

```
    def __init__(self, width, height, spi, dc, res, cs, external_vcc=False):
        self.rate = 10 * 1024 * 1024
        dc.init(dc.OUT, value=0)
        res.init(res.OUT, value=0)
        cs.init(cs.OUT, value=1)
        self.spi = spi
        self.dc = dc
        self.res = res
        self.cs = cs
        import time
```

```
        self.res(1)
        time.sleep_ms(1)
        self.res(0)
        time.sleep_ms(10)
        self.res(1)
        super().__init__(width, height, external_vcc)
```

```
    def write_cmd(self, cmd):
        self.spi.init(baudrate=self.rate, polarity=0, phase=0)
        self.cs(1)
        self.dc(0)
        self.cs(0)
        self.spi.write(bytearray([cmd]))
        self.cs(1)
```

```
def write_data(self, buf):
    self.spi.init(baudrate=self.rate, polarity=0, phase=0)
    self.cs(1)
    self.dc(1)
    self.cs(0)
    self.spi.write(buf)
    self.cs(1)
```

Código para leer texto y adaptarlo para las dimensiones del Display usando una clase “oled.py”

Se genero este código en base a una clase; que contabiliza el numero de caracteres que tiene la cadena en algún texto que se vaya a mandar de la ESP al Display, una vez que toma eso, dimensiona el tamaño de la pantalla y ajusta el texto para que no aparezca cortado o incompleto el texto. Tambien se agrego una función para que cada que detecte un ‘\n’ haga un salto de línea en el display y siga escribiendo el texto deseado, se invoca a esta clase con ‘from oled import Display’

```
# Desplegando texto en Display de forma ordenada como CLASE
# RILB
# Ahora reconoce palabras grandes y las pasa a la siguiente linea
# Ahora con solo poner \n puedes hacer saltos de linea
from machine import Pin, I2C
from ssd1306 import SSD1306_I2C

class Display:
    def __init__(self, width=128, height=64, scl_pin=22, sda_pin=21):
        self.width = width
        self.height = height
        self.i2c = I2C(0, scl=Pin(scl_pin), sda=Pin(sda_pin))
        self.display = SSD1306_I2C(width, height, self.i2c)
        self.line_height = 10
        self.chars_per_line = self.width // 8
        self.max_lines = self.height // self.line_height

    def clear(self):
        self.display.fill(0)
        self.display.show()

    def show(self):
        self.display.show()

    def text(self, message, x=0, y=0):
        self.display.fill(0)
        lines = []
        for raw_line in message.split("\n"):
            words = raw_line.split()
            current_line = ""
            for word in words:
                if len(current_line + " " + word) <= self.chars_per_line:
```

```

        if current_line:
            current_line += " " + word
        else:
            current_line = word
    else:
        lines.append(current_line)
        current_line = word
    if current_line:
        lines.append(current_line)

    for i, line in enumerate(lines[:self.max_lines]):
        self.display.text(line, x, y + i * self.line_height)

    self.display.show()

```

Código para ESP-4 “esp_now_ver4.py”

```

# ESP-NOW Comunicación entre 4 ESP32 (Topología Cuadrado) con Display, 2 Push Buttons y ACK
# Cada nodo envía mensajes de entre 3 opciones predefinidas y reenvía (solo una vez) para que todos los
# nodos puedan leerlos.
# Se evita retroalimentación infinita usando el prefijo "FWD:" en mensajes reenviados.
# Cambiamos el mensaje "ACK:" por "Recibio??:\n"
# RILB

from machine import Pin
from oled import Display
import network
import espnow
import time

# ----- CONFIGURACIÓN DEL NODO -----
# Define el ID de este nodo (cámbialo a "ESP1", "ESP2", "ESP3" o "ESP4" según corresponda)
DEVICE_ID = "ESP4" # Ejemplo: este nodo es ESP4

# Define las MAC reales de cada nodo (reemplaza estos valores por los tuyos)
peers = {
    "ESP1": b'\x9c\x9c\x1f\xc8\x07\xc4',      # ---- de Joel
    "ESP2": b'\x08\xb6\x1f7\xd6\\',          # ---- de LAB
    "ESP3": b'\xa0\xb7eb\xef\xa4'             # ---- de LYF
    #"ESP4": b'\x94<\xc62\xca0'               # ---- de RILB
}

# Configuración de conectividad según topología (cuadrado):
# Ejemplo de topología:
#  ESP1 ----- ESP2
#  |             |
#  ESP4 ----- ESP3
neighbors_ids = {

```



```

"ESP1": ["ESP2", "ESP3", "ESP4"],
"ESP2": ["ESP1", "ESP3", "ESP4"],
"ESP3": ["ESP2", "ESP4", "ESP1"],
"ESP4": ["ESP1", "ESP3", "ESP2"]
# Se debe actualizar por si se agregan nuevas ESP, la configuración de ahora es ESP1 tiene a ESP2, ESP3 y
ESP4 como vecinos; dependiendo de cómo este montado
}
# Lista de vecinos (MACs) para este nodo:
neighbors = [peers[n] for n in neighbors_ids[DEVICE_ID]]

# Mapeo de alias: se usa para mostrar nombres cortos en el display.
alias = {peers[k]: k for k in peers if k != DEVICE_ID} # no se incluye el propio nodo

# ----- INICIALIZACIÓN WIFI y ESP-NOW -----
w0 = network.WLAN(network.STA_IF)
w0.active(True)

# Inicializa ESP-NOW
esp = espnow.ESPNow()
esp.active(True) # Activa ESP-NOW

# Agrega a ESP-NOW solo los peers vecinos (no se agrega el propio)
for mac in neighbors:
    esp.add_peer(mac)

# ----- MENSAJES PREDEFINIDOS -----
mensajes = ["Holi", "me reporto", "AYUDAAA!"]
indice_msg = 0

# ----- DISPLAY OLED -----
display = Display()
def mostrar_menu():
    display.text("Mensaje actual:\n{}".format(mensajes[indice_msg]))

display.text("{} listo".format(DEVICE_ID))
time.sleep(2)
mostrar_menu()

# ----- CONFIGURACIÓN DE LOS BOTONES -----
# Se usan dos push buttons: uno para cambiar el mensaje y otro para enviarlo.
# Ajusta los pines según tus conexiones.
boton_cambiar = Pin(2, Pin.IN, Pin.PULL_UP) # Botón para cambiar mensaje
boton_enviar = Pin(4, Pin.IN, Pin.PULL_UP) # Botón para enviar mensaje

ultimo_estado_cambiar = boton_cambiar.value()
ultimo_estado_enviar = boton_enviar.value()

# ----- ENVÍO CON ACK -----
def enviar_mensaje():
    global indice_msg

```

```
# Componer mensaje: "DEVICE_ID:mensaje"
```

```
msg_text = "{}:{}".format(DEVICE_ID, mensajes[indice_msg])
```

```
display.text("Enviando:\n{}".format(mensajes[indice_msg]))
```

```
# Reinicia el diccionario de ACK para cada vecino
```

```
ack_status = {mac: False for mac in neighbors}
```

```
# Envía el mensaje a todos los vecinos
```

```
for mac in neighbors:
```

```
    esp.send(mac, msg_text.encode())
```

```
# Espera 1 segundo para recolectar ACKs
```

```
start_ack = time.time()
```

```
while time.time() - start_ack < 1000:
```

```
    if esp.any():
```

```
        host, rec = esp.recv()
```

```
        rec_text = rec.decode()
```

```
        if rec_text.startswith("Recibio??:\n"):
```

```
            # Registro de ACK: se asume que el ACK es "ACK:remitente"
```

```
            ack_status[host] = True
```

```
        time.sleep(0.05)
```

```
# Construye el resumen de recepción
```

```
resumen = "Recibio??:\n"
```

```
for mac in neighbors:
```

```
    nombre = alias.get(mac, mac.hex()[2:8])
```

```
    if ack_status.get(mac, False):
```

```
        resumen += " {} Si".format(nombre)
```

```
    else:
```

```
        resumen += " {} NO".format(nombre)
```

```
display.text(resumen)
```

```
time.sleep(2)
```

```
mostrar_menu()
```

```
# ----- BUCLE PRINCIPAL -----
```

```
while True:
```

```
    estado_cambiar = boton_cambiar.value()
```

```
    estado_enviar = boton_enviar.value()
```

```
    if estado_cambiar == 0 and ultimo_estado_cambiar == 1:
```

```
        # Cambia el mensaje (incrementa el índice)
```

```
        indice_msg = (indice_msg + 1) % len(mensajes)
```

```
        mostrar_menu()
```

```
        time.sleep(0.3)
```

```
    if estado_enviar == 0 and ultimo_estado_enviar == 1:
```

```
        enviar_mensaje()
```

```
    ultimo_estado_cambiar = estado_cambiar
```

```
    ultimo_estado_enviar = estado_enviar
```

Procesar recepción de mensajes

```
if esp.any():
    host, rec = esp.recv()
    rec_text = rec.decode()
    # Si el mensaje recibido es un ACK, se ignora en esta sección.
    if not rec_text.startswith("Recibio??:\n"):
        # Se asume que el mensaje es "SENDER:contenido"
        parts = rec_text.split(":", 1)
        if len(parts) == 2:
            sender, contenido = parts
        else:
            sender, contenido = "?", rec_text

    # Si el mensaje proviene de este mismo nodo, lo ignora para evitar bucles.
    if sender == DEVICE_ID:
        # No se procesa el mensaje para evitar que ESP2 procese su propio mensaje reenviado.
        pass
    else:
        # Si el mensaje no tiene el prefijo "FWD:" (es original), lo reenviamos a los vecinos (excepto al que lo envió)
        if not rec_text.startswith("FWD:"):
            forward_text = "FWD:" + rec_text
            for mac in neighbors:
                if mac != host:
                    esp.send(mac, forward_text.encode())
        # Muestra el mensaje recibido en el display
        sender_alias = alias.get(host, host.hex()[:8])
        display.text("{}:\n{}".format(sender_alias, contenido))
        time.sleep(2)
        mostrar_menu()
        # Envía ACK de vuelta al remitente
        ack_msg = "Recibio??:\n" + DEVICE_ID
        esp.send(host, ack_msg.encode())
```

Código para ESP-3 “esp_now_ver4_2.py”

Este es el código para controlar el menú de envío con un Joystick, manteniendo casi de la misma forma el código con los botones.

```
# ESP-NOW Comunicación entre 4 ESP32 (Topología Cuadrado) con Display, Joystick y ACK
# RILB

from machine import Pin, ADC
from oled import Display
import network
import espnow
import time
```

```

# ----- CONFIGURACIÓN DEL NODO -----
# Define el ID de este nodo (cámbialo a "ESP1", "ESP2", "ESP3" o "ESP4" según corresponda)
DEVICE_ID = "ESP3" # Ejemplo: este nodo es ESP3

# Define las MAC reales de cada nodo (reemplaza estos valores por los tuyos)
peers = {
    "ESP1": b'\x9c\x9c\x1f\xc8\x07\xc4',          #----de Joel
    "ESP2": b'\x08\xb6\x1f\xd6\\',              #----de LAB
    "ESP3": b'\xa0\xb7eb\xef\xa4',              #----de Lucero
    "ESP4": b'\x94<\xc62\xca0'                  #----de RILB
}

# Configuración de conectividad según topología (cuadrado):
# Ejemplo:
#   ESP1 ----- ESP2
#   |           |
#   ESP4 ----- ESP3
neighbors_ids = {
    "ESP1": ["ESP2", "ESP4"],
    "ESP2": ["ESP1", "ESP3"],
    "ESP3": ["ESP2", "ESP4"],
    "ESP4": ["ESP1", "ESP3"]
}

# Lista de vecinos (MACs) para este nodo:
neighbors = [peers[n] for n in neighbors_ids[DEVICE_ID]]

# Mapeo de alias: se usa para mostrar nombres cortos en el display.
alias = {peers[k]: k for k in peers if k != DEVICE_ID} # no se incluye el propio nodo

# ----- INICIALIZACIÓN WIFI y ESP-NOW -----
w0 = network.WLAN(network.STA_IF)
w0.active(True)

# Inicializa ESP-NOW
esp = espnow.ESPNow()
esp.active(True) # Activa ESP-NOW

# Agrega a ESP-NOW solo los peers vecinos (no agregamos el propio)
for mac in neighbors:
    esp.add_peer(mac)

# ----- MENSAJES PREDEFINIDOS -----
mensajes = ["Holi", "me reporto", "AYUDAAA!"]
indice_msg = 0

# ----- DISPLAY OLED -----
display = Display()
def mostrar_menu():
    display.text("Mensaje actual:\n{}".format(mensajes[indice_msg]))

```

```

display.text("{} listo".format(DEVICE_ID))
time.sleep(2)
mostrar_menu()

# ----- CONFIGURACIÓN DEL JOYSTICK -----
# Se usará el eje X (ADC) y el botón del joystick.
joystick_x = ADC(Pin(32)) # Eje X en GPIO32-ADC1
joystick_btn = Pin(2, Pin.IN, Pin.PULL_UP) # Botón en GPIO2

# Umbral para detectar movimiento (ajusta según tu joystick)
umbral_izq = 3000
umbral_der = 4500

last_move = time.ticks_ms()
last_btn = joystick_btn.value()

# ----- ENVÍO CON ACK -----
def enviar_mensaje():
    global indice_msg
    # Componer mensaje: "DEVICE_ID:mensaje"
    msg_text = "{}:{}".format(DEVICE_ID, mensajes[indice_msg])
    display.text("Enviando:\n{}".format(mensajes[indice_msg]))

    # Reinicia el diccionario de ACK para cada vecino
    ack_status = {mac: False for mac in neighbors}

    # Envía el mensaje a todos los vecinos
    for mac in neighbors:
        esp.send(mac, msg_text.encode())

    # Espera 2 segundos para recolectar ACKs
    start_ack = time.ticks_ms()
    while time.ticks_diff(time.ticks_ms(), start_ack) < 1000:
        if esp.any():
            host, rec = esp.recv()
            rec_text = rec.decode()
            if rec_text.startswith("ACK:"):
                # Registro de ACK: se asume que el ACK es "ACK:remitente"
                ack_status[host] = True
            time.sleep(0.05)

    # Construye el resumen de recepción
    resumen = "Recibio??:\n"
    for mac in neighbors:
        nombre = alias.get(mac, mac.hex()[:8])
        if ack_status.get(mac, False):
            resumen += " {} Si".format(nombre)
        else:
            resumen += " {} NO".format(nombre)

```

```
display.text(resumen)
```

```
time.sleep(2)
```

```
mostrar_menu()
```

```
# ----- BUCLE PRINCIPAL -----
```

```
while True:
```

```
    # Lectura del eje X para cambiar mensaje
```

```
    x_val = joystick_x.read()
```

```
    ahora = time.ticks_ms()
```

```
    if time.ticks_diff(ahora, last_move) > 300:
```

```
        if x_val < umbral_izq:
```

```
            indice_msg = (indice_msg - 1) % len(mensajes)
```

```
            mostrar_menu()
```

```
            last_move = ahora
```

```
        elif x_val > umbral_der:
```

```
            indice_msg = (indice_msg + 1) % len(mensajes)
```

```
            mostrar_menu()
```

```
            last_move = ahora
```

```
    # Botón del joystick para enviar mensaje
```

```
    btn_val = joystick_btn.value()
```

```
    if btn_val == 0 and last_btn == 1:
```

```
        enviar_mensaje()
```

```
    last_btn = btn_val
```

```
    # Procesar recepción de mensajes
```

```
    if esp.any():
```

```
        host, rec = esp.recv()
```

```
        rec_text = rec.decode()
```

```
        # Si se recibe un ACK, se procesa en la función de envío; si es un mensaje normal:
```

```
        if not rec_text.startswith("ACK:"):
```

```
            # Se asume que el mensaje es "SENDER:contenido"
```

```
            parts = rec_text.split(":", 1)
```

```
            if len(parts) == 2:
```

```
                sender, contenido = parts
```

```
            else:
```

```
                sender, contenido = "?", rec_text
```

```
        # Muestra el mensaje recibido
```

```
        sender_alias = alias.get(host, host.hex()[:8])
```

```
        display.text("{}:\n{}".format(sender_alias, contenido))
```

```
        time.sleep(2)
```

```
        mostrar_menu()
```

```
        # Envía ACK de vuelta al remitente
```

```
        ack_msg = "ACK:" + DEVICE_ID
```

```
        esp.send(host, ack_msg.encode())
```

```
    time.sleep(0.1)
```

Código para ESP-2 “esp_now_ver4_LAB.py”

Este código se uso para la optimización de espacio ya que al solo tener un protoboard disponible, se tuvieron que hacer algunos cambios en las entradas y salidas de los pines, más aparte tener que conectar por la parte inferior de la ESP, algunos cables que condijeran a los pines faltantes.

```
# ESP-NOW Comunicación entre 4 ESP32 (Topología Cuadrado) con Display, 2 Push Buttons y ACK
# Cada nodo envía mensajes de entre 3 opciones predefinidas y reenvía (solo una vez) para que todos los
# nodos puedan leerlos.
# Se evita retroalimentación infinita usando el prefijo "FWD:" en mensajes reenviados.
# RILB

from machine import Pin
from oled import Display
import network
import espnow
import time

# ----- CONFIGURACIÓN DEL NODO -----
# Define el ID de este nodo (cámbialo a "ESP1", "ESP2", "ESP3" o "ESP4" según corresponda)
DEVICE_ID = "ESP2" # Ejemplo: este nodo es ESP2

# Define las MAC reales de cada nodo (reemplaza estos valores por los tuyos)
peers = {
    "ESP1": b'\x9c\x9c\x1f\xc8\x07\xc4',    # ---- de Joel
    "ESP2": b'\x08\xb6\x1f7\xd6\\',          # ---- de LAB
    "ESP3": b'\xa0\xb7eb\xef\xa4',          # ---- de Lucero
    "ESP4": b'\x94<\xc62\xca0'              # ---- de RILB
}

# Configuración de conectividad según topología (cuadrado):
# Ejemplo:
#   ESP1 ----- ESP2
#   |           |
#   ESP4 ----- ESP3
neighbors_ids = {
    "ESP1": ["ESP2", "ESP3", "ESP4"],
    "ESP2": ["ESP1", "ESP3", "ESP4"], # ESP2 tiene como vecinos a ESP1, ESP3 y ESP4
    "ESP3": ["ESP2", "ESP4", "ESP1"],
    "ESP4": ["ESP1", "ESP3", "ESP2"]
}

# Lista de vecinos (MACs) para este nodo:
neighbors = [peers[n] for n in neighbors_ids[DEVICE_ID]]

# Mapeo de alias: se usa para mostrar nombres cortos en el display.
alias = {peers[k]: k for k in peers if k != DEVICE_ID} # No se incluye el propio nodo

# ----- INICIALIZACIÓN WIFI y ESP-NOW -----
w0 = network.WLAN(network.STA_IF)
w0.active(True)
```

```

# Inicializa ESP-NOW
esp = espnow.ESPNow()
esp.active(True) # Activa ESP-NOW

# Agrega a ESP-NOW solo los peers vecinos (no se agrega el propio)
for mac in neighbors:
    esp.add_peer(mac)

# ----- MENSAJES PREDEFINIDOS -----
mensajes = ["Holi", "me reporto", "AYUDAAA!"]
indice_msg = 0

# ----- DISPLAY OLED -----
display = Display()
def mostrar_menu():
    display.text("Mensaje actual:\n{}".format(mensajes[indice_msg]))

display.text("{} listo".format(DEVICE_ID))
time.sleep(2)
mostrar_menu()

# ----- CONFIGURACIÓN DE LOS BOTONES -----
# Se usan dos push buttons: uno para cambiar el mensaje y otro para enviarlo.
# Ajusta los pines según tus conexiones.
boton_cambiar = Pin(16, Pin.IN, Pin.PULL_UP) # Botón para cambiar mensaje
boton_enviar = Pin(4, Pin.IN, Pin.PULL_UP) # Botón para enviar mensaje

ultimo_estado_cambiar = boton_cambiar.value()
ultimo_estado_enviar = boton_enviar.value()

# ----- ENVÍO CON ACK -----
def enviar_mensaje():
    global indice_msg
    # Componer mensaje: "DEVICE_ID:mensaje"
    msg_text = "{}:{}".format(DEVICE_ID, mensajes[indice_msg])
    display.text("Enviando:\n{}".format(mensajes[indice_msg]))

    # Reinicia el diccionario de ACK para cada vecino
    ack_status = {mac: False for mac in neighbors}

    # Envía el mensaje a todos los vecinos
    for mac in neighbors:
        esp.send(mac, msg_text.encode())

    # Espera 1 segundo para recolectar ACKs
    start_ack = time.ticks_ms()
    while time.ticks_diff(time.ticks_ms(), start_ack) < 1000:
        if esp.any():
            host, rec = esp.recv()

```



```
rec_text = rec.decode()
if rec_text.startswith("Recibio??:\n"):
    ack_status[host] = True
time.sleep(0.05)
```

```
# Construye el resumen de recepción
resumen = "Recibio??:\n"
for mac in neighbors:
    nombre = alias.get(mac, mac.hex()[8:])
    if ack_status.get(mac, False):
        resumen += " {} Si".format(nombre)
    else:
        resumen += " {} NO".format(nombre)
display.text(resumen)
time.sleep(2)
mostrar_menu()
```

```
# ----- BUCLE PRINCIPAL -----
```

```
while True:
    estado_cambiar = boton_cambiar.value()
    estado_enviar = boton_enviar.value()

    if estado_cambiar == 0 and ultimo_estado_cambiar == 1:
        # Cambia el mensaje (incrementa el índice)
        indice_msg = (indice_msg + 1) % len(mensajes)
        mostrar_menu()
        time.sleep(0.3)
```

```
    if estado_enviar == 0 and ultimo_estado_enviar == 1:
        enviar_mensaje()
```

```
    ultimo_estado_cambiar = estado_cambiar
    ultimo_estado_enviar = estado_enviar
```

```
# Procesar recepción de mensajes
```

```
if esp.any():
    host, rec = esp.recv()
    rec_text = rec.decode()
    # Si el mensaje recibido es un ACK, se ignora en esta sección.
    if not rec_text.startswith("Recibio??:\n"):
        # Se asume que el mensaje es "SENDER:contenido"
        parts = rec_text.split(":", 1)
        if len(parts) == 2:
            sender, contenido = parts
        else:
            sender, contenido = "?", rec_text
```

```
# Si el mensaje proviene de este mismo nodo, lo ignora para evitar bucles.
```

```
if sender == DEVICE_ID:
```

```
    # No se procesa el mensaje para evitar que ESP2 procese su propio mensaje reenviado.
```

```
pass
else:
    # Si el mensaje no tiene el prefijo "FWD:" (es original), lo reenviamos a los vecinos (excepto al que
    lo envió)
    if not rec_text.startswith("FWD:"):
        forward_text = "FWD:" + rec_text
        for mac in neighbors:
            if mac != host:
                esp.send(mac, forward_text.encode())
    # Muestra el mensaje recibido en el display
    sender_alias = alias.get(host, host.hex()[:8])
    display.text("{}:\n{}".format(sender_alias, contenido))
    time.sleep(2)
    mostrar_menu()
    # Envía ACK de vuelta al remitente
    ack_msg = "Recibio??:\n" + DEVICE_ID
    esp.send(host, ack_msg.encode())
```

Bibliografía.

Espressif Systems – Documentación oficial de ESP-NOW

https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html

Random Nerd Tutorials – Guías sobre ESP-NOW y ESP32

<https://randomnerdtutorials.com/esp-now-esp32-arduino-ide/>

Adafruit Learning System – Uso de pantallas OLED con MicroPython

<https://learn.adafruit.com/micropython-hardware-ssd1306-oled-display>